Operating Systems: Model Exam Answers

Professor's Note: Each question is worth 3 marks. Answers must be precise. Keywords are bolded for emphasis.

1. Explain why an operating system is referred to as a "resource manager"?

The Operating System (OS) is called a resource manager because it acts as the arbitrator between conflicting requests from programs and users for computer resources.

- **Multiplexing:** It manages resources in two ways: in **time** (different programs take turns using the CPU) and in **space** (different programs get different chunks of memory).
- **Allocation:** It decides which process gets which resource (CPU, RAM, I/O devices), when they get it, and for how long.
- **Reclamation:** It ensures resources are reclaimed (freed) after a process finishes to prevent memory leaks or deadlocks.

2. Differentiate between CLI and GUI.

Feature	CLI (Command Line Interface)	GUI (Graphical User Interface)
Interaction	User types text commands (e.g., `ls` , `mkdir`).	User interacts via visual elements (icons, windows, menus).
Efficiency	**High efficiency** for experienced users; consumes fewer system resources.	**Lower efficiency** for complex tasks; consumes high memory/CPU overhead.
Ease of Use	Steep learning curve; requires memorization of syntax.	User-friendly; intuitive for beginners.

3. Explain how an operating system handles interrupts.

An interrupt is a signal to the processor to stop its current activity and deal with a high-priority event. The handling process involves:

1. **Context Saving:** The OS pauses the currently executing process and saves its state (Program Counter, registers) onto the stack or PCB.
2. **Interrupt Service Routine (ISR):** The OS looks up the interrupt number in the **Interrupt Vector Table** to find the location of the specific code (ISR) needed to handle that event.
3. **Restore:** Once the ISR executes, the OS restores the saved context of the previous process and resumes execution exactly where it left off.

4. Compare FCFS (First-Come, First-Served) and Round Robin Scheduling.

1. **Preemption:** FCFS is **non-preemptive** (once a process starts, it runs to completion). Round Robin is **preemptive** (processes are stopped after a specific time quantum).

2. **Response Time:** FCFS has poor response time, leading to the **Convoy Effect** (short jobs wait behind long jobs). Round Robin provides excellent response time, making it ideal for time-sharing systems.
3. **Fairness:** FCFS is fair in order of arrival but unfair in processing time. Round Robin is fair in CPU allocation, giving every process an equal slice of time.

5. Analyze the role of PCB (Process Control Block).

The PCB is a data structure acting as the "brain" or specific representation of a process in the OS. Its role is critical for **Context Switching**:

- **State Management:** It stores the Process State (New, Ready, Running, Waiting, Terminated).
- **Context Preservation:** It holds the Program Counter (address of the next instruction) and CPU registers so the process can resume correctly after an interruption.
- **Accounting & Management:** It contains scheduling information (priority), memory limits, and a list of open files/I/O devices associated with the process.

6. Explain any two types of system calls supported by the Operating System.

System calls provide the interface between a running program and the kernel.

1. **Process Control:** These calls manage process creation and termination.
   - *Example:* `fork()` creates a new process; `exit()` terminates a process.
2. **File Management:** These calls handle file manipulation.
   - *Example:* `open()` to access a file; `read()` / `write()` to modify content; `close()` to release the file handle.

7. What is meant by Kernel in Operating System?

The Kernel is the central core of the operating system.

- **Residency:** It is the portion of the OS that resides in the main memory (RAM) at all times during computer operation.
- **Privilege:** It executes in **Kernel Mode** (privileged mode), allowing it direct access to hardware and memory that standard user applications cannot touch.
- **Function:** It handles the lowest-level abstraction layers: memory management, process scheduling, and hardware interfacing.

8. Mention any two services provided by Operating System.

1. **Program Execution:** The OS loads a program into memory (RAM), allocates resources, initializes execution, and handles program termination (normal or error-based).
2. **I/O Operations:** Since user programs cannot execute I/O operations directly (for security), the OS provides a uniform interface to read/write to devices (disks, printers, networks) via drivers.

9. Discuss the evolution of Operating System.

- **Serial Processing (1940s-50s):** No OS; programmers interacted directly with hardware using toggle switches. Inefficient.
- **Batch Processing (1950s-60s):** Similar jobs were grouped (batched) to speed up setup time. CPU was often idle during I/O.
- **Multiprogramming & Time-Sharing (1960s-Present):** Introduction of holding multiple jobs in memory to keep CPU busy (Multiprogramming) and rapid switching to support multiple interactive users (Time-Sharing).

10. What is multiprogramming? Explain its concept in the context of OS.

Multiprogramming increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute.

- **Concept:** Several jobs are kept in main memory simultaneously.
- **Mechanism:** When the currently running job needs to wait (e.g., for I/O), the OS switches the CPU to another job in the ready queue.
- **Goal:** To minimize CPU idle time and maximize system throughput.

11. Describe the function and importance of device driver in Operating System.

- **Function:** A device driver is a specialized software module that controls a specific type of hardware device attached to a computer. It translates general OS instructions into specific device signals.
- **Importance:** It provides **abstraction**. The OS does not need to know the complex details of how a specific hard drive or printer works; it just calls the standard driver functions. This allows new hardware to be added without rewriting the OS kernel.

12. What do you understand by an operating system?

An Operating System is system software that manages computer hardware and software resources and provides common services for computer programs.

- **Interface:** It acts as an intermediary between the user and the computer hardware.
- **Control:** It controls and coordinates the use of hardware among various application programs.
- **Examples:** Windows, Linux, macOS, Android.

13. List any three major functions of an operating system.

1. **Process Management:** Creating, scheduling, and terminating processes (programs in execution).
2. **Memory Management:** Tracking which parts of memory are in use, allocating memory to processes, and de-allocating it when done.

3. **File Management:** Creating, deleting, and manipulating files and directories; mapping files to physical storage.

14. Name any three types of operating systems and briefly describe each.

1. **Batch OS:** Jobs are processed in groups without user interaction (e.g., Payroll systems).
2. **Real-Time OS (RTOS):** Requires processing within rigid time constraints. Used in critical systems (e.g., Air traffic control).
3. **Distributed OS:** Manages a group of distinct computers and makes them appear to be a single computer to the user (e.g., Cloud computing platforms).

15. Identify the component that acts as an interface between the user and computer hardware and briefly explain its role.

The **Operating System** (specifically the Shell or GUI layer) is the component.

- **Role:** It hides the complexities of the hardware (abstraction) from the user. Instead of writing binary code to spin a hard drive disk, the user simply clicks "Save." The OS translates this high-level request into the low-level machine instructions required by the hardware.

16. What is a system call?

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system.

- It serves as the boundary between the **User Space** (where applications run) and the **Kernel Space** (where the OS runs).
- It is triggered via a software interrupt (trap).

17. Name one real-time operating system and mention one application area.

- **Name:** VxWorks (or FreeRTOS, QNX).
- **Application Area:** Industrial Robotics (or Missile Guidance Systems, Medical Pacemakers). *Note: The application must require strict timing where a delay causes system failure.*

18. Define time-sharing operating system.

A time-sharing OS allows many users to share the computer resources simultaneously.

- **Mechanism:** It uses CPU scheduling (usually Round Robin) and multiprogramming to provide each user with a small slice of time.
- **Illusion:** The switching is so rapid that each user feels they have the entire computer to themselves, even though it is being shared.

19. Calculate turnaround time if waiting time is 4 and burst time is 6.

- **Formula:** Turnaround Time (TAT) = Waiting Time (WT) + Burst Time (BT).
- **Calculation:** TAT = 4 + 6.

- **Answer:** Turnaround Time = **10 units**.

20. Apply SJF (Shortest Job First) scheduling for burst time 2, 4, 6.

Assumption: All processes arrive at Time 0.

Processes:

- P1: Burst Time = 2
- P2: Burst Time = 4
- P3: Burst Time = 6

Execution Order (SJF selects smallest burst first):

1. **P1** runs first (0 to 2).
2. **P2** runs second (2 to 6).
3. **P3** runs third (6 to 12).

Gantt Chart: `| P1 (0-2) | P2 (2-6) | P3 (6-12) |`

Waiting Times: P1=0, P2=2, P3=6. **Average Waiting Time:** (0+2+6)/3 = 2.66 units